Campus Free Slot Finder

Project Proposal and System Design

Integrated project scope | 16 October 2026

Project Overview

Campus Free Slot Finder is a web application that combines official college timetable data with existing reservations to help students, faculty and clubs make informed use of campus spaces. It brings together three connected functions: finding free slots, checking a teacher's scheduled venue, and requesting venues for club or event activities.

The system checks timetable entries, dated bookings, closures and room policies before showing availability. A blank timetable cell is not automatically treated as proof that a room is free. Where timetable coverage or room details are incomplete, the result is marked **Unknown** instead of making an unsupported promise.

Core Features

Feature	What it does
1. Free Slot Finder	Searches a selected date and time window for available classrooms, laboratories and other venues, checking timetable entries and existing reservations to prevent clashes.
2. Teacher Venue Finder	Searches a teacher by name and shows their scheduled classes, listed classroom/lab/lecture theatre, free periods and next class when timetable data is available. It does not show real-time physical location.
3. Club and Event Venue Booking	Lets clubs check venues such as KP Nautiyal, Open Ground and New Auditorium, select a date and time, submit a request, and track or cancel bookings where policy allows.

Booking and Approval Model

Students and faculty can submit requests. For spaces governed by the allocation workflow, only faculty explicitly assigned responsibility for that room or building can approve or reject a request. Approval rechecks the latest availability and creates the reservation in a single database transaction. Pending requests do not reserve a room, and a conflicting request cannot be approved.

Implementation Status and Scope

The supplied timetable extraction has been exercised and retained as draft data for review. The application, booking backend, teacher search, approval workflow and operational controls described in this proposal are implementation requirements, not claims that the complete system is already built or tested.

The first release targets one campus and supported timetable layouts. Room identities, capacity, term dates, elective membership and complete timetable coverage must be verified before affected slots can be confirmed.

Core Features and User Workflows

The three features share the same timetable and booking records. This avoids treating a room as free in one screen while another timetable or reservation already occupies it.

1. Free Slot Finder

Users choose a date, start time, end time or required duration, and the type of space they need. The system compares the requested window with published classes, confirmed reservations, closures and room restrictions.

- Search for available classrooms, labs, lecture theatres and other venues.
- Check a specific room or compare suitable options by time, capacity and required equipment.
- Detect partial overlaps as well as bookings that cover the entire requested window.
- Show the result as Available, Occupied, Restricted or Unknown, with reasons and timetable freshness where available.
- Do not treat a search result as a reservation; a booking is confirmed only through the configured approval process.

2. Teacher Venue Finder

Users search for a teacher by name and select a day or time. The system reads the teacher's timetable and returns the scheduled class and the venue recorded for that class.

- View a teacher's timetable for a selected day or time.
- Display the classroom, laboratory or lecture theatre listed against a scheduled class.
- Show free periods and the next scheduled class when verified timetable records are available.
- If a class, venue or timetable mapping is missing or ambiguous, show that limitation rather than guessing.

Important: the result represents the teacher's scheduled venue according to timetable records, not their real-time physical location.

3. Club and Event Venue Booking

Clubs can check venues such as KP Nautiyal, Open Ground, New Auditorium and other registered college spaces for meetings, workshops and events.

- Select the venue, date, start time, end time, purpose, expected attendance and equipment needs.
- Check for timetable use, closures and existing reservations during any part of the requested interval.
- Submit a request and track whether it is Pending, Approved, Rejected, Withdrawn or Expired.
- Only the assigned allocation faculty for that space can approve or reject; rejection records a reason.
- Approval rechecks availability immediately before reserving the room. If another request wins the race, the conflicting request stays pending with a conflict message.
- View booking history and cancel a confirmed reservation when the user or responsible allocator is permitted to do so.

How the Features Work Together

The Free Slot Finder provides the availability check, the Teacher Venue Finder reads the same verified timetable records, and Club and Event Venue Booking adds reservation records to that shared schedule. Confirmed bookings, classes and closures all participate in conflict checks. Changes to official timetable versions are reviewed before publication so that an updated class schedule cannot silently overwrite an existing reservation.

Roles at a Glance

Role	Main responsibility
Student / faculty requester	Search spaces, submit requests, track decisions and manage own bookings under policy.
Allocation faculty	Review requests only for assigned rooms/buildings; approve or reject with a reason.
System administrator	Manage accounts, room inventory, allocation assignments, timetable imports and publication.

System-wide rule: if the required timetable coverage, room mapping or participant information is incomplete, the system must not present the slot as verified free.

1 Users requests and allocation faculty

Room allocation has two distinct functions: requesting space and deciding who may use it. Both students and faculty can request. Only faculty explicitly assigned the allocation role for the selected space can approve or reject. An individual may hold both roles, but cannot approve their own request.

Role	Permitted actions
Student requester	Search permitted spaces, submit requests, track decisions, withdraw pending requests and cancel own confirmed reservations under policy.
Faculty requester	The same request flow, including extra classes, meetings and academic events. Faculty status does not bypass approval.
Allocation faculty	Receive the inbox for assigned rooms or buildings, review requests, approve or reject with a reason, and manage allocations within that scope.
System administrator	Maintain accounts, allocation responsibilities, inventory and timetable versions. Approval needs a separate allocation assignment.

Request information and routing

Collect purpose, selected room, date, start and end, expected attendance, required equipment and relevant participant groups. A supervisor may be required by room policy; a teacher is not mandatory for every student study session. Participant availability is checked only for the declared, verified participants; unknown membership is never presented as verified.

On submission, the server maps the room to its responsible allocation group. Assigned allocation faculty receive a durable in-app inbox item and an outbox-backed notification; the requester sees Pending approval. Only eligible faculty receive the request. Unmapped spaces cannot accept requests and are flagged to the system administrator for assignment.

Decision workflow

Pending requests do not hold a room, so several can coexist. Approval checks the latest database state, verifies allocation authority and creates the booking in one transaction. Rejection requires a reason. If the room became unavailable, approval fails and the request stays pending with a conflict message; an alternative requires requester acceptance before approval.

States: Pending may become Approved, Rejected, Withdrawn or Expired. Approval links exactly one booking. Later cancellation updates the booking and occupancy atomically while retaining the approved decision history. Approved means reserved; Pending means awaiting a decision.

Inbox reliability and concurrent decisions

GET /requests/mine shows requester history; POST /requests/{id}/withdraw withdraws a pending request. Allocation faculty use the scoped inbox. Lock the request row before any decision, withdrawal or expiry; a unique request-to-booking link prevents two allocators approving the same request twice. Scope changes reroute pending work and are audited. The first committed valid decision wins.

2 Evidence from the supplied timetable

The uploaded B.Tech-C.S.E-V-TT-2-2.pdf has 18 pages with native text and drawn table lines. Its metadata reports creator Word and a Quartz PDF producer on macOS, with creation date 1 October 2026. This identifies the export path; it does not establish which tool originally computed the schedule. All pages state an effective date of 13 July 2026. [1]

Verified finding	Result
Section pages	A1, A2, B1, B2, C1, C2, D1, D2, E1, E2, F1, F2, G1, CS1, ML1, ML2, ML3, DS1
Grid shape	Five weekdays and ten time columns per page
Populated timetable cells	455 retained as draft records
Merged lab sessions	95 continuous sessions spanning two printed periods
Elective cells	72 mention ELECTIVE; 57 contain multiple course alternatives
Distinct room label candidates	22 labels; building identity and eligibility still need confirmation

Verified example

A1 Tuesday PCS 502 occupies 302-A (LAB) from 14:10 to 16:00. Its merged cell must produce one interval. The PDF also places A1 Tuesday PCS 514 in 102-B (LAB) from 16:00 to 17:50. Those times are read from the column geometry, not guessed from text ordering.

Differences from the screenshot

The current PDF places A1 CEC in 106 (LT), whereas the earlier screenshot lists the auditorium. The Machine Learning elective legend also changes its room and faculty. Therefore the screenshot and PDF must remain separate versions; mixing their cells would create an internally inconsistent schedule. [1, 2]

What remains unknown

This file does not establish a term end date, authoritative room capacities, all building identities, holiday exceptions, complete campus room usage or elective membership. Some faculty rows contain alternative names. These are publication gates, not fields to fill with guesses. The extracted JSON remains a draft.

Native extraction avoids OCR recognition errors, but it does not remove semantic ambiguity. Representative visual checks and structural checks were completed; the 455 cells are not claimed to have received an independent human audit.

3 Import pipeline and reusable timetable format

The canonical exchange format is JSON validated against JSON Schema Draft 2020-12. The companion package contains an empty template, a schema, draft data from all 18 pages, original grid and legend extraction, and a validator. The format is general; a PDF adapter remains specific to a supported layout.

Layer	Purpose and core fields
Document	schema_version, institution_id, timezone, term_id, timetable_version, valid_from, valid_until, status
Source evidence	source_id, filename, SHA-256 digest, page count; each session retains page and cell bounding box
Resource registry	resource_id, kind, label, mapping_status, building_id, capacity, features
Weekly session	session_id, day_of_week, start_local, end_local, course_code, kind, group_ids, room_ids, teacher_ids
Review information	resolution, issues, raw_text, source_refs and unresolved elective_options
Period metadata	Optional named periods for display; actual start and end times remain authoritative

Import steps

Accept an administrator upload; validate file type and size, calculate its digest and preserve the original. A separate Python worker extracts text and geometry with pdfplumber. It identifies the header and day rows, maps each cell to time boundaries, expands merged cells, and extracts the subject legend. It then applies explicit aliases and validates the resulting JSON. [3]

Preserve 017 and 002-A as strings. Normalize TCS 597(E) to TCS597E, but do not merge it with TCS597. Infer afternoon times only under this confirmed GEHU adapter convention; preserve the printed text. A different layout must declare its own time convention.

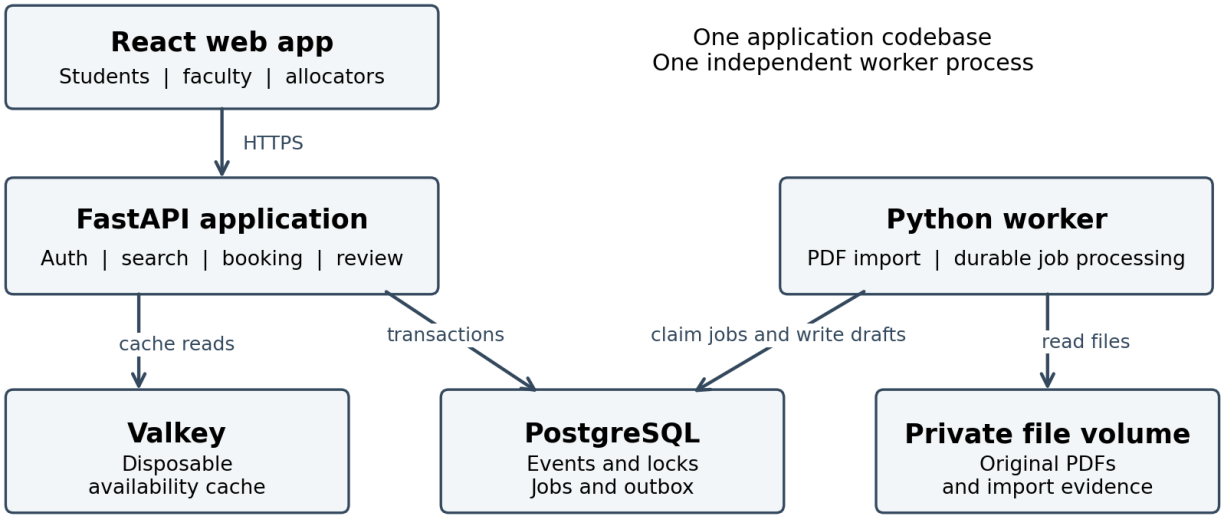
Review and publication gates

Show each candidate beside its source cell. Verify room identities, faculty assignments, term dates and group membership. Expand elective alternatives into separate real events. Link confirmed shared classes across section pages to one event with multiple source references. Matching room and time alone is insufficient proof that two records are duplicates.

A null grid entry covered by a merged cell is not free time. A blank cell means no listed session for that section. Missing sections, unknown rooms and unparsed cells make affected availability unknown. Unexpected layouts stop import rather than silently dropping content.

4 Fixed technology stack and architecture

Component	Selected technology	Responsibility
Web application	React with TypeScript and Vite	Search, requests, allocation inbox and timetable administration
Backend	Python 3.12 with FastAPI and Pydantic	Validated APIs and one modular application
Data access	SQLAlchemy 2 and Alembic	Transactions and schema migrations
Authoritative database	PostgreSQL 18 with btree_gist	Events, constraints, locks, versions and audit
Cache	Self-hosted Valkey with valkey-py	Disposable availability results and fill coordination
Import processing	pdfplumber in a Python worker	Native text and geometry extraction; no LLM
Job processing	PostgreSQL job and outbox tables	Durable worker leases and retry tracking
Deployment	Docker Compose and Caddy on Linux	HTTPS, app, worker, database and cache containers
Files and testing	Private persistent volume; pytest and Playwright	Source retention, API and browser verification



The API writes uploads to the private volume and records import jobs in PostgreSQL. The worker claims jobs from the database; no separate queue service is required. Start with one API service and one worker from the same codebase. [4, 5]

Valkey is the selected BSD-licensed open-source cache. Self-hosting needs no managed-cache subscription, although hosting infrastructure still has a cost. Pin tested package versions and container digests before deployment. C++ custom HTTP serving, Spring Boot and SQLite are removed from this revised stack. [9]

Campus Free Sl

5 Data model and availability calculation

Entity family	Key relationships and invariants
Identity and inventory	Campus owns users, roles and resources. Scoped allocation assignments link designated faculty to rooms or buildings; rooms carry capacity, equipment and opening hours.
Timetable evidence	Import owns immutable source files, extracted candidates and validation issues. Published versions retain effective dates and source references.
Recurring schedule	Weekly rules expand into dated class events through a confirmed finite term horizon, excluding holidays and applying approved exceptions.
Events and occupancy	An approved allocation request creates one booking event. Each event has occupancy rows for its required room and participants. Classes, bookings and closures share conflict checks.
Coverage and policies	Track verified schedule coverage by resource and date, calendar coverage and permission rules. Incomplete coverage prevents automatic confirmation.
Coordination records	resource_day_state holds revisions and lock rows; idempotency_request deduplicates commands; audit and outbox track durable consequences.

Resource identities and student groups

Use stable database resource IDs scoped by campus. A full section and one of its lab subgroups cannot be treated as unrelated resources. Expand attendees into a set of non-overlapping membership groups: a full-section session reserves every constituent group; a subgroup session reserves its members. Elective combinations may require finer membership groups. Until that mapping is verified, offer room-only results and mark participant availability unknown.

Availability algorithm

Filter rooms by permissions, capacity and equipment. For each eligible room and participant set, collect dated busy intervals and restrictions, add required setup or travel buffers, sort and merge overlaps, then subtract them from the permitted search window. Intersect the remaining intervals across required resources and keep windows long enough for the requested duration. This costs O(n log n) for sorting n retrieved intervals, before ranking.

Use half-open intervals [start,end): an event ending at 11:05 does not overlap one starting at 11:05. For intervals a and b, overlap means a.start < b.end AND b.start < a.end. Store concrete occurrences as timestamptz; derive them from campus-local dates and times. Reject unsupported overnight input in the MVP.

Rank valid options by earliest start, suitable capacity, required equipment and preferred building. Show the factors rather than an unexplained score. Report Available, Occupied, Restricted or Unknown, with source coverage and an as-of time. No availability display is a reservation.

6 Database integrity and concurrent readers

The readers-writers problem appears when many users inspect availability while others book rooms or replace timetables. Ordinary searches use PostgreSQL MVCC snapshots rather than a global application mutex. Readers can see a committed snapshot while writers perform short transactions. A read-only REPEATABLE READ transaction gives a multi-query search a consistent view. [6]

Atomicity alone is not sufficient

Two transactions can both read an empty interval and then attempt to insert. Wrapping the code in BEGIN and COMMIT does not by itself prevent that race. Correctness comes from a database exclusion constraint plus a shared writer protocol. All application writers, worker jobs and administrative mutations must obey it.

Database enforced non overlap

For each dated event, create occupancy rows for the room, teacher and atomic attendee groups. Use a half-open tstzrange and the following exclusion rule. btree_gist supplies equality support for the scalar identifiers. The complete reference DDL is in the format package. [7]

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
-- Inside occupancy table definition
during tstzrange GENERATED ALWAYS AS
  (tstzrange(starts_at, ends_at, '[)')) STORED,
CHECK (starts_at < ends_at),
EXCLUDE USING gist (
  campus_id WITH =,
  resource_id WITH =,
  during WITH &&
) WHERE (active)
```

Classes and reservations must reach this same table. Separate unchecked tables for each would allow cross-table clashes. On cancellation, change the event status and every occupancy active flag in one transaction. Direct writes are restricted to the application role; user requests cannot toggle occupancy flags.

Concrete race example

A student and a faculty member submit requests for the same verified-free room and time. Both may remain pending. When allocation faculty try to approve them, the approval transactions contend on the same room/date state row. After the first approval commits, the second rechecks the database and receives HTTP 409; its request stays pending for an alternative. Even if a writer omits the friendly overlap check, the exclusion constraint rejects a conflicting occupancy insert.

The constraint covers time overlap. Capacity, coverage, authorization, hierarchy and version consistency still require the transaction checks described next. A uniqueness rule on room and start time is inadequate because partially overlapping intervals can start at different times.

7 Booking transaction and specific lock keys

Use PostgreSQL READ COMMITTED for write transactions, with explicit locks and exclusion constraints. Booking creation is internal to approval; requesters have no direct booking endpoint. All state-changing scheduling operations use the primary database. Keep parsing, cache calls, notifications and network requests outside these transactions.

Purpose	Key and acquisition rule
Publication guard	PostgreSQL advisory pair (41001, campus_id). Booking and cancellation use pg_advisory_xact_lock_shared; publication and policy or membership changes use pg_advisory_xact_lock.
Resource day serialization	Row key (campus_id, resource_id, local_date) in resource_day_state. Create missing rows, then SELECT FOR UPDATE in ascending key order.
Duplicate command protection	Unique database key (campus_id, user_id, request_key). Retain a SHA-256 hash of the canonical request and its committed response.
Lock lifetime	Transaction-scoped locks release at commit or rollback. Initial limits: lock_timeout 2 seconds, statement_timeout 8 seconds; measure and tune.

Write protocol

1. Authenticate the allocation faculty member. Begin the transaction and take the shared campus guard. A publisher or allocation-assignment change uses its exclusive counterpart.

2. Reserve the idempotency key with INSERT ON CONFLICT. An in-flight duplicate waits; a committed duplicate returns the stored response. Reject reuse with a different payload.

3. Lock the allocation request by (campus_id, request_id). Verify pending status, expected request revision, current allocation scope and no self-approval. Resolve resources server-side; create and lock resource/day rows in sorted order, including dates touched by buffers.

4. Recheck current coverage, active timetable, authorization, opening hours, equipment, capacity, participant membership and overlapping occupancy after locks are held.

5. Insert the unique booking for that request and all occupancy rows; mark the request approved with decided_by and decided_at. Increment data_rev, append audit and outbox records, and store the idempotent response in the same transaction.

6. Commit, then return success. An insert failure rolls back every partial allocation. A handled domain conflict may use a savepoint to persist an idempotent 409 response without retaining partial rows.

Lock order is campus guard, idempotency key, request row, then sorted resource/day rows. Withdrawal, rejection and expiry take the request lock too. Never lock only matching occupancy rows: an empty slot has no row to lock. Never acquire per-resource locks in request order. Avoid shared-to-exclusive guard upgrades. Retry deadlock or serialization failures with bounded jitter, at most three attempts and the same idempotency key; return a conflict for exclusion violations and a retryable response for lock timeout. [8]

8 Valkey cache and consistency strategy

Valkey stores derived availability, not authoritative reservations. A cache hit speeds up searching; it never authorizes a booking. If Valkey is unavailable, calculate from PostgreSQL with bounded request concurrency. Keep availability usable without making the cache a single correctness dependency. [9]

Keys and lifetimes

Use	Exact key pattern and value
Resource availability	avail:v1:{campus}:{resource}:{date}:s{S}:c{C}:r{R}. Value contains busy intervals, restrictions, coverage status and generated_at. TTL randomly 25-35 seconds.
Cache fill coordination	fill:v1:{campus}:{resource}:{date}:s{S}:c{C}:r{R}. Random ownership token; SET key token NX PX 2000.
Database revisions	S = campus schedule_rev; C = campus catalog_rev; R = resource/day data_rev. Read all required revisions together from the primary database.
Idempotency	Kept in PostgreSQL, not Valkey, so eviction or cache loss cannot recreate a previously accepted booking.

Read and refresh protocol

Start a short read-only REPEATABLE READ transaction. Fetch active schedule/catalog revisions, coverage and resource/day revisions in a batched query; a missing day row has revision zero. Read only the corresponding versioned cache keys. On a miss, calculate from the same database snapshot and write under those captured versions. End the transaction promptly; a cache timeout falls back to the database.

Submitting a pending request does not change occupancy or availability revisions. Approval or cancellation increments R atomically. Timetable publication increments S; inventory, policy or membership changes increment C under the exclusive campus guard. Coverage or calendar changes must bump the revision that governs them. Old keys expire naturally. A slow reader can only populate an old-version key, so it cannot overwrite a newer version with stale intervals.

Stampedes and lock ownership

Only the holder of a fill key normally performs the cache refresh. Other readers wait briefly with jitter or calculate directly under a bounded concurrency limit. Release the fill key with a Lua compare-and-delete operation, never an unconditional DEL. If its two-second lease expires, duplicate calculations are acceptable: this lock is a performance optimization, not a booking lock. [10]

Outbox and failure recovery

Request submission and a durable allocation-inbox entry commit with an outbox event. Approval, rejection and booking changes also commit their notifications through the outbox. A worker leases pending outbox rows, processes notifications and optional cache cleanup, and records completion. Expired leases are retried. Delivery is at least once; consumers deduplicate by outbox ID. Do not claim exactly-once delivery. Database-read revisions keep cache correctness independent of delayed cleanup.

9 Timetable publication and exception handling

Draft staging

Uploading never mutates the active schedule. The import worker writes a new draft version with its source hash, adapter version, row counts, unresolved fields and original-cell evidence. Re-uploading identical content reuses the existing import for that campus and scope. Changed bytes create a new version even when the filename stays the same.

Prepublication validation

An administrator supplies the term end date and confirms resource mappings, faculty assignments, membership and calendar exceptions. Expand recurring rules into dated occurrences for the entire allowed booking horizon. Validate new events against one another and against bookings, other active timetable scopes and closures. A global campus schedule revision can represent several independently published section or semester scopes.

Atomic activation

Precompute the change set outside the lock. In a short transaction, take the exclusive campus advisory guard, re-read the base revision and repeat conflict validation. Booking writers that already hold the shared guard finish first; subsequent writers wait. If the draft was built against an obsolete revision, reject and rebuild the change set.

Deactivate only the replaced scope and effective dates, insert the replacement occupancies, switch the active version, increment schedule_rev, and commit audit/outbox records together. Preserve historical events and unaffected scopes. If any check fails, roll back the entire change; readers continue to see a complete committed version.

Conflict policy

Default policy: do not publish a new timetable that collides with a confirmed reservation. List the affected booking, room, date and source cell. An authorized administrator must explicitly cancel or reschedule those bookings, with a reason and notification, before retrying publication. There is no silent overwrite. A shared event referenced by several scopes cannot be removed while another active scope still depends on it.

Calendar and service exceptions

- Holidays and closures are dated restrictions. A class cancellation affects only the chosen occurrence unless the administrator explicitly changes the recurring rule.
- Every student or faculty request needs designated allocation-faculty approval. Pending requests do not reserve space. Approval rechecks availability and may fail if another request was approved first.
- A room or teacher cancellation uses the same resource locks, revisions and audit protocol as booking. Rescheduling releases the old allocation and acquires the new one atomically.
- If the official website cannot be fetched, retain the last verified version and display its freshness. Allow administrator upload; do not clear occupancy because a fetch failed.

10 APIs permissions and operational boundaries

Endpoint	Role	Behavior
POST /imports	Administrator	Upload PDF or canonical JSON; return import job ID with HTTP 202.
GET /imports/{id}	Administrator	Return progress, source evidence and validation issues.
PATCH /imports/{id}/sessions/{sid}	Administrator	Correct draft fields; require If-Match draft revision to prevent lost edits.
POST /imports/{id}/publish	Administrator	Require expected base revision; validate and atomically activate.
GET /availability	Student or faculty	Accept date/window, duration, room needs and participant IDs; return status and reasons.
POST /requests	Student or faculty	Require Idempotency-Key; create a pending request, routed inbox item and notification. No room hold.
GET /allocation/inbox	Allocation faculty	Return pending requests for currently assigned spaces only.
POST /bookings/{id}/cancel	Owner or scoped allocator	Authorize, lock and cancel with reason; repeated cancellation is harmless.
POST /requests/{id}/approve or /reject	Allocation faculty	Approve atomically after rechecking availability, or reject with a reason. Enforce scope and no self-approval.

Authorization and privacy

Derive campus and user identity from the authenticated session, not request body values. Enforce ownership, current allocation scope and request status on every object lookup. Administrator status alone does not confer approval permission. Use Argon2id password hashes for the MVP, opaque server-side sessions stored in PostgreSQL, Secure HttpOnly SameSite cookies and CSRF checks on mutations. Do not store bearer credentials in browser local storage. Campus SSO is a later integration.

Keep source PDFs and teacher details private to authorized roles. Student-facing views should reveal only the information they need. Use parameterized SQL, upload byte/page limits, parser time and memory limits, and a non-executing worker environment. If URL import is added, restrict fetches to configured official hosts and revalidate redirects to prevent internal-network requests.

Deployment and recovery

Run Caddy, web/API, worker, PostgreSQL and Valkey with Docker Compose. Only HTTPS is public; database and cache ports remain private. Persist database and upload volumes. Use environment-managed secrets, migration backups and a tested restore procedure. Valkey cache contents can be discarded after restart.

Record request IDs, import duration, unresolved-cell counts, booking conflict rate, database lock wait, cache hit rate, job age and outbox retries. Monitor coverage and materialization horizon. PostgreSQL outage stops confirmations with a clear retryable response; cached results must not be presented as newly verified availability.

11 Expected problems and concrete remedies

Problem	Failure mode	Required remedy
Concurrent readers and writers	A search sees a schedule changing midway.	MVCC; one read-only snapshot for multi-query searches, version-tagged results.
Concurrent writers	Two allocation approvals target the same empty slot.	Resource/day row locks plus a PostgreSQL exclusion constraint and atomic insert.
Phantom interval	Locking existing bookings locks nothing for a free room.	Create a stable resource_day_state row; lock it before checking occupancy.
Deadlocks	Requests acquire rooms or participants in opposite order.	One lock hierarchy and sorted resource/date keys; bounded retries.
Stale cache refill	An older reader repopulates cache after a booking.	Database revision included in the key; no unversioned latest-result overwrite.
Duplicate submissions	A retry duplicates a request or approval.	Persistent idempotency key, canonical payload hash and stored response.
Partial failure	Room booked but teacher/group allocation fails.	All occupancy rows, audit, revision and outbox changes share one transaction.
Publish versus booking	A new class replaces a slot while it is being reserved.	Shared/exclusive campus transaction guard and validation under the guard.
Missing or ambiguous schedules	Unknown data is presented as free.	Coverage registry; unresolved imports stay draft; affected results are Unknown.
Section or elective overlap	A subgroup is booked during its parent section class.	Server-side expansion to disjoint attendee groups; verify membership.
Shared timetable entries	One shared class is mistaken for multiple conflicts.	Review canonical shared-event mapping; retain all source references.
Cache or worker failure	Search stops or notifications vanish.	Database fallback; durable jobs/outbox with leases and idempotent consumers.

Bookings require known room suitability and verified schedule coverage for the requested resources and dates. These checks are business invariants: cache speed or an overlap-free query cannot substitute for them. When a whole department timetable is missing, limit the verified scope rather than declaring all unlisted rooms free.

12 Validation plan and hackathon demonstration

Acceptance tests below are implementation gates, not completed test results. Run concurrency tests against real PostgreSQL connections; mocks and a single-process mutex cannot demonstrate cross-instance correctness.

Test	Expected result
PDF fixture extraction	18 section grids; 455 populated cells retained; 95 merged labs; explicit review issues; no silently discarded cells.
Schema and semantic validation	Draft validates; unknown references, invalid dates, wrong resource types and reversed times fail. Publication rejects the supplied unresolved draft.
100 competing same-slot approvals	On an eligible empty slot, exactly one distinct request confirms. Others conflict or receive a retryable timeout; retries resolve to conflicts. Check database rows.
Partial overlaps and boundaries	Overlapping starts fail; adjacent [10:10,11:05) and [11:05,12:00) pass when buffer rules allow.
Teacher and subgroup conflict	Different rooms still conflict when they require the same teacher or overlapping attendees.
Two allocators decide one request	Request-row lock and unique request-to-booking link permit one decision and at most one booking, even with different idempotency keys.
Idempotent retry after timeout	A committed request returns its original response; no second event or duplicate outbox side effect.
Publication race and stale edit	Writer and publisher cannot produce incompatible active state. Stale base and draft revisions are rejected.
Stale fill and Valkey outage	Old refill cannot satisfy a new revision key. Search falls back to PostgreSQL; booking behavior remains correct.
Crash and cancellation	Precommit crash leaves no partial booking; postcommit retry is safe; cancellation releases all resources atomically.
Authorization and isolation	Unauthorized imports, self-approval, out-of-scope decisions and cross-campus access are rejected. Student and faculty request flows both succeed.

Performance targets to measure

Initial demo targets: p95 search under 500 ms for a one-day query over 100 rooms, and uncontended booking under one second on declared hardware. Test with 50 simultaneous search users, separately from the contention test. Record dataset size, cache state, hardware, p50/p95 latency and lock waits. These are goals, not measured results.

Demonstration sequence

Upload the real PDF and show a merged lab plus a flagged elective. Resolve a small, clearly labeled pilot scope with verified room and participant records. Submit one student request and one faculty request for the same room. Show them in the responsible allocation-faculty inbox, then attempt competing approvals. Show one confirmed booking and one pending conflict. Publish a revised timetable that collides with it and demonstrate the explicit resolution workflow. Finally disable Valkey and show database-backed operation.

13 Delivery plan and references

Implementation milestones

Milestone	Exit condition
Import foundation	Canonical JSON, parser adapter, source provenance and review screen preserve every candidate.
Trusted schedule	Verified inventory, membership, finite calendar and pilot coverage; dated occupancies materialized.
Safe booking	Student/faculty requests, scoped approval, constraints, locks, idempotency and cancellation pass concurrent tests.
Performance and updates	Valkey versioned cache, durable worker and timetable replacement pass failure tests.
Submission readiness	Recorded demo, measured results, deployment instructions and clear scope limitations.

Corrections to the original proposal

Use the single stack in Section 4 throughout. Students and faculty both request; designated allocation faculty receive and decide requests. Replace generic data-structure claims with interval merging, indexed overlap queries and explicit database constraints. Describe cancellation with authorization and an audit trail. Separate demonstrated import results from planned application features. Treat timetable completeness, publication safety and concurrency tests as core work. [2]

Source references

[1] Supplied B.Tech-C.S.E-V-TT-2-2.pdf, 18 pages. Local text, geometry and metadata inspection on 16 October 2026. Public source: https://btechcsegehu.in/wp-content/uploads/2026/10/B.Tech-C.S.E-V-TT-2.pdf

[2] Supplied Daa PHASE 3_removed.pdf and timetable screenshot image(20261016-025306).png. Used for proposal corrections and version comparison.

[3] pdfplumber official repository: https://github.com/jsvine/pdfplumber

[4] FastAPI official documentation: https://fastapi.tiangolo.com/

[5] SQLAlchemy transaction management: https://docs.sqlalchemy.org/en/20/orm/session_transaction.html

[6] PostgreSQL MVCC and isolation: https://www.postgresql.org/docs/18/mvcc-intro.html and https://www.postgresql.org/docs/18/transaction-iso.html

[7] PostgreSQL ranges and exclusion constraints: https://www.postgresql.org/docs/18/rangetypes.html

[8] PostgreSQL explicit locks and advisory functions: https://www.postgresql.org/docs/18/explicit-locking.html and https://www.postgresql.org/docs/18/functions-admin.html

[9] Valkey project and clients: https://valkey.io/ and https://valkey.io/clients/

[10] Valkey SET and Lua evaluation: https://valkey.io/commands/set/ and https://valkey.io/commands/eval/

The concurrency protocol and cache key design are proposed application decisions built on these documented primitives. They require the acceptance tests above before production use.